Pimpri Chinchwad Education Trust's

Pimpri Chinchwad College of Engineering

Department of Information Technology

ASSIGNMENT 4

NAME	Viraj Shrikant Pathare
PRN	124B1F149
CLASS	SY IT
DIVISION	A

Schema Overview (appraisal_system database):

```
-- Employee, Department, Appraisal, Goals, Training, Salary_History tables
-- (Refer Assignment 2 for full CREATE TABLE statements)
```

Stored Procedures

Stored Procedures encapsulate reusable SQL logic that can accept parameters, perform DML operations, and return result sets.

Procedure 1 — Register New Employee

Inserts a new employee record. Validates that the email is not already registered before inserting. Raises a custom error if a duplicate email is detected.

```
DELIMITER $$
CREATE PROCEDURE sp_RegisterEmployee (
IN p_name VARCHAR(100),
IN p_email VARCHAR(150),
IN p_phone VARCHAR(15),
IN p_dept_id INT,
IN p_designation VARCHAR(80),
IN p_salary DECIMAL(10,2),
OUT p_emp_id INT
)
BEGIN
IF EXISTS (SELECT 1 FROM Employee WHERE email = p_email) THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Email already registered.';
ELSE
INSERT INTO Employee (name, email, phone, dept_id, designation,
joining_date, salary)
VALUES (p_name, p_email, p_phone, p_dept_id, p_designation,
CURDATE(), p_salary);
SET p_emp_id = LAST_INSERT_ID();
END IF;
END$$
DELIMITER ;

-- Usage:
CALL sp_RegisterEmployee('Divya Reddy','divya@company.com',
'9900112233', 1, 'Junior Developer', 45000, @new_id);
SELECT @new_id AS newly_created_emp_id;
```

Sample Output:

```
newly_created_emp_id
--------------------
6
```

Procedure 2 — Submit Appraisal

Creates a new appraisal record for an employee. Validates that the reviewer is not the same as the employee being appraised. Automatically records salary hike if rating >= 4.5.

```
DELIMITER $$
CREATE PROCEDURE sp_SubmitAppraisal (
IN p_emp_id INT,
IN p_reviewer_id INT,
IN p_rating DECIMAL(3,1),
IN p_comments TEXT
)
BEGIN
DECLARE v_salary DECIMAL(10,2);
DECLARE v_new_sal DECIMAL(10,2);

IF p_emp_id = p_reviewer_id THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Employee cannot review themselves.';
END IF;

INSERT INTO Appraisal (emp_id, reviewer_id, appraisal_date,
rating, comments, status)
VALUES (p_emp_id, p_reviewer_id, CURDATE(),
p_rating, p_comments, 'Completed');

-- Auto salary hike for outstanding rating
IF p_rating >= 4.5 THEN
SELECT salary INTO v_salary FROM Employee WHERE emp_id = p_emp_id;
SET v_new_sal = v_salary * 1.10; -- 10% hike
UPDATE Employee SET salary = v_new_sal WHERE emp_id = p_emp_id;
INSERT INTO Salary_History (emp_id, old_salary, new_salary,
effective_date, reason)
VALUES (p_emp_id, v_salary, v_new_sal, CURDATE(),
CONCAT('Auto hike - Rating ', p_rating));
END IF;

SELECT LAST_INSERT_ID() AS appraisal_id, p_rating AS rating,
'Appraisal submitted.' AS message;
END$$
DELIMITER ;
```

```
-- Output
CALL sp_SubmitAppraisal(1, 2, 4.7, 'Exceptional delivery this cycle.');
```

Sample Output:

```
appraisal_id | rating | message
-------------|--------|--------------------
6 | 4.7 | Appraisal submitted.
```

Procedure 3 — Assign Goal to Employee

Adds a new performance goal for an employee. Validates that target date is in the future.

```
DELIMITER $$
CREATE PROCEDURE sp_AssignGoal (
IN p_emp_id INT,
IN p_description VARCHAR(255),
IN p_target_date DATE
)
BEGIN
IF p_target_date <= CURDATE() THEN
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Target date must be in the future.';
END IF;
```

```
INSERT INTO Goals (emp_id, description, target_date, achieved)
VALUES (p_emp_id, p_description, p_target_date, 0);

SELECT LAST_INSERT_ID() AS goal_id,
'Goal assigned successfully.' AS message;
END$$
DELIMITER ;

-- Usage:
CALL sp_AssignGoal(2, 'Reduce attrition by 15%', '2025-12-31');
```

Sample Output:

```
goal_id | message
--------|----------------------------
6 | Goal assigned successfully.
```

Procedure 4 — Approve Appraisal

Updates appraisal status to Approved. Validates current status is Completed before approval.

```
DELIMITER $$
CREATE PROCEDURE sp_ApproveAppraisal (
IN p_appraisal_id INT,
OUT p_result VARCHAR(100)
)
BEGIN
DECLARE v_status VARCHAR(20);
SELECT status INTO v_status FROM Appraisal
WHERE appraisal_id = p_appraisal_id;

IF v_status IS NULL THEN
SET p_result = 'Appraisal not found.';
ELSEIF v_status = 'Approved' THEN
SET p_result = 'Already approved.';
ELSEIF v_status = 'Pending' THEN
SET p_result = 'Cannot approve - still Pending.';
ELSE
UPDATE Appraisal SET status = 'Approved'
WHERE appraisal_id = p_appraisal_id;
SET p_result = 'Appraisal approved successfully.';
END IF;
END$$
DELIMITER ;

-- Usage:
CALL sp_ApproveAppraisal(1, @msg);
SELECT @msg AS result;
```

Sample Output:

```
result
------------------------------
Appraisal approved successfully.
```

Procedure 5 — Get Employee Performance Report

Generates a comprehensive performance report by joining Employee, Department, Appraisal, Training, and Goals.

```
DELIMITER $$
CREATE PROCEDURE sp_EmployeeReport (IN p_emp_id INT)
BEGIN
SELECT e.emp_id, e.name, e.email,
d.dept_name, e.designation, e.salary,
a.appraisal_date, a.rating, a.status AS appraisal_status,
a.comments,
COUNT(DISTINCT t.training_id) AS total_trainings,
AVG(t.score) AS avg_training_score,
COUNT(DISTINCT g.goal_id) AS total_goals,
SUM(g.achieved) AS goals_achieved
```

```
FROM Employee e
JOIN Department d ON e.dept_id = d.dept_id
LEFT JOIN Appraisal a ON e.emp_id = a.emp_id
LEFT JOIN Training t ON e.emp_id = t.emp_id
LEFT JOIN Goals g ON e.emp_id = g.emp_id
WHERE e.emp_id = p_emp_id
GROUP BY e.emp_id, a.appraisal_id;
END$$
DELIMITER ;

-- Usage:
```

Sample Output:

```
1 | Rahul Patil | rahul@company.com | Engineering | Software Engineer
| 65000.00 | 2024-12-31 | 4.2 | Completed | Excellent performance...
| total_trainings: 2 | avg_score: 90.25 | goals: 2 | achieved: 2
```

User-Defined Functions (UDFs)

UDFs return a single computed value and can be embedded directly inside SELECT, WHERE, or HAVING clauses.

Function 1 — fn_GetAppraisalGrade()

Returns a letter grade (A+/A/B/C/D) based on a numeric appraisal rating.

```
DELIMITER $$
CREATE FUNCTION fn_GetAppraisalGrade (p_rating DECIMAL(3,1))
RETURNS VARCHAR(5)
DETERMINISTIC
BEGIN
RETURN CASE
WHEN p_rating >= 4.5 THEN 'A+'
WHEN p_rating >= 4.0 THEN 'A'
WHEN p_rating >= 3.5 THEN 'B'
WHEN p_rating >= 3.0 THEN 'C'
ELSE 'D'
END;
END$$
DELIMITER ;

-- Usage:
SELECT e.name, a.rating,
fn_GetAppraisalGrade(a.rating) AS grade
FROM Employee e
JOIN Appraisal a ON e.emp_id = a.emp_id;
```

Sample Output:

```
+--------------+--------+     +
| name | rating | grade |
+--------------+--------+     +
| Rahul Patil | 4.2 | A |
| Sneha Sharma | 4.8 | A+ |
| Amit Verma | 3.5 | B |
| Pooja Nair | 4.0 | A |
| Karan Mehta | 3.8 | B |
+--------------+--------+     +
```

Function 2 — fn_CalculateBonus()

Calculates the annual bonus amount for an employee based on their appraisal rating and current salary.

```
DELIMITER $$
CREATE FUNCTION fn_CalculateBonus (
p_emp_id INT,
p_rating DECIMAL(3,1)
)
RETURNS DECIMAL(12,2)
DETERMINISTIC READS SQL DATA
BEGIN
DECLARE v_salary DECIMAL(10,2) DEFAULT 0;
DECLARE v_pct DECIMAL(5,2) DEFAULT 0;

SELECT salary INTO v_salary FROM Employee WHERE emp_id = p_emp_id;

SET v_pct = CASE
WHEN p_rating >= 4.5 THEN 0.20
WHEN p_rating >= 4.0 THEN 0.15
WHEN p_rating >= 3.5 THEN 0.10
WHEN p_rating >= 3.0 THEN 0.05
ELSE 0.00
END;

RETURN ROUND(v_salary * v_pct, 2);
END$$
DELIMITER ;

-- Usage:
```

```
SELECT e.emp_id, e.name, a.rating,
fn_CalculateBonus(e.emp_id, a.rating) AS annual_bonus
FROM Employee e JOIN Appraisal a ON e.emp_id = a.emp_id;
```

Sample Output:

```
| Rahul Patil | 4.2 | 9750.00 |
| Sneha Sharma | 4.8 | 18000.00 |
| Amit Verma | 3.5 | 7500.00 |
| Pooja Nair | 4.0 | 9000.00 |
| Karan Mehta | 3.8 | 6000.00 |
```

Function 3 — fn_GetGoalCompletion()

Returns the goal completion percentage for a given employee.

```
DELIMITER $$
CREATE FUNCTION fn_GetGoalCompletion (p_emp_id INT)
RETURNS DECIMAL(5,2)
DETERMINISTIC READS SQL DATA
BEGIN
DECLARE v_total INT DEFAULT 0;
DECLARE v_achieved INT DEFAULT 0;

SELECT COUNT(*), SUM(achieved)
INTO v_total, v_achieved
FROM Goals WHERE emp_id = p_emp_id;

IF v_total = 0 THEN RETURN 0.00; END IF;
RETURN ROUND((v_achieved / v_total) * 100, 2);
END$$
DELIMITER ;

-- Usage:
SELECT u.emp_id, u.name,
fn_GetGoalCompletion(u.emp_id) AS goal_pct
FROM Employee u;
```

Sample Output:

```
| 1 | Rahul Patil | 100.00 |
| 2 | Sneha Sharma | 100.00 |
| 3 | Amit Verma | 0.00 |
| 4 | Pooja Nair | 100.00 |
| 5 | Karan Mehta | 0.00 |
```

Function 4 — fn_GetYearsOfService()

Returns the number of years an employee has been with the company.

```
DELIMITER $$
CREATE FUNCTION fn_GetYearsOfService (p_emp_id INT)
RETURNS INT
DETERMINISTIC READS SQL DATA
BEGIN
DECLARE v_join DATE;
SELECT joining_date INTO v_join FROM Employee WHERE emp_id = p_emp_id;
RETURN TIMESTAMPDIFF(YEAR, v_join, CURDATE());
END$$
DELIMITER ;

-- Usage:
SELECT name, joining_date,
fn_GetYearsOfService(emp_id) AS years_of_service
FROM Employee;
```

Sample Output:

```
| Rahul Patil | 2022-01-10 | 3 |
| Sneha Sharma | 2020-03-15 | 5 |
| Amit Verma | 2019-07-01 | 5 |
| Pooja Nair | 2021-05-20 | 4 |
```

```
| Karan Mehta | 2023-02-01 | 2 |
```

Function 5 — fn_GetAvgTrainingScore()

Returns the average training score for a given employee. Returns 0 if no training exists.

```
DELIMITER $$
CREATE FUNCTION fn_GetAvgTrainingScore (p_emp_id INT)
RETURNS DECIMAL(5,2)
DETERMINISTIC READS SQL DATA
BEGIN
DECLARE v_avg DECIMAL(5,2) DEFAULT 0;
SELECT COALESCE(AVG(score), 0) INTO v_avg
FROM Training WHERE emp_id = p_emp_id;
RETURN v_avg;
END$$
DELIMITER ;

-- Combined employee dashboard:
SELECT e.emp_id, e.name,
fn_GetAppraisalGrade(a.rating) AS grade,
fn_CalculateBonus(e.emp_id, a.rating) AS bonus,
fn_GetGoalCompletion(e.emp_id) AS goal_pct,
fn_GetYearsOfService(e.emp_id) AS years,
fn_GetAvgTrainingScore(e.emp_id) AS avg_score
FROM Employee e
JOIN Appraisal a ON e.emp_id = a.emp_id
ORDER BY a.rating DESC;
```

Sample Output:

```
| Sneha Sharma | A+ | 18000.00 | 100.00 | 5 | 95.00 |
| Rahul Patil | A | 9750.00 | 100.00 | 3 | 90.25 |
| Pooja Nair | A | 9000.00 | 100.00 | 4 | 85.00 |
| Karan Mehta | B | 6000.00 | 0.00 | 2 | 0.00 |
| Amit Verma | B | 7500.00 | 0.00 | 5 | 80.00 |
```